

3. Create the manifest file based on the *package.json* as per the guidelines in nodered.org/docs/creating-nodes/packaging.html. Then add *eventsSource* as a dependency and replace "sample": "sample/sample.js" with "opensors": "opensors/opensors.js".
4. Edit *opensors.html* as follows. Under *RED.nodes.registerType*, make zero inputs and one output, as this node doesn't receive any inputs and only generates payloads whenever server side events are triggered.

```
inputs: 0, output:1
```

We'll consider the pre-existing topic field as the *opensors* topic, and the name is the default field kept for all nodes.

5. Edit *opensors.js* as follows. Add the following lines in the beginning, in place of the *foo* entry:

```
var EventSource = require("eventsSource");
var myurl=require('url');
```

After *respond* to *inputs*....remove the *this.on* ("input",); and add the following code:

```
var url=myurl.parse("https://realtime.opensors.io/
v1/public/events/topics/"+this.topic);
var eventUrl = new EventSource(url);
var msg={};
eventUrl.on('message',function(event) {
    msg.payload=event.data;
    node.send(msg);
}); //or use any callback style of node.js
```

Rename *SampleNode* as *OpenSensorsNode* and replace all occurrences of *sample* with *opensors* in both JavaScript and HTML files.

That's it—the custom node is ready to use now. Restart *Node-RED* and refresh the browser page. You can find the new node under the input category, drag it and connect it to the debug node. Fill a suitable topic name in the *Edit* panel and deploy the flow. You will observe that all the published messages on this topic are captured by this node. If you are not planning to create an account initially, try the same with */orgs/osfydemo/rpisense/weather*, a public topic created for this example.

 **Note:** If you want to modify this node for private topics, add another text box for the API key with the entry *node-input-apikey* (similar coding of topic, name fields), and a checkbox with the entry *node-input-ispublic* along with the following code:

```
<div class="form-row">
<label for="node-input-ispublic">
<input id="node-input-ispublic" type="checkbox">
    public topic </label> </div>
```

Add suitable initial values under defaults section in html file

```
apikey: { value:"xxxx"},
ispublic : {value:true, required:true}
```

Note that a comma must be added at the end of the first line of the above code. Now, in js, type the following code in the local copy section:

```
this.apikey = n.apikey;
this.ispublic = n.ispublic;
```

Next, form the url as follows:

```
if(this.ispublic)
url=myurl.parse("https://realtime.opensors.io
/v1/public/events/topics/"+this.topic);
else
url=myurl.parse("https://realtime.opensors.io/v1/events
/topics/"+this.topic+"?api-key="+this.apikey);
```

You can compare your code with the example code *node-red-contrib-osfydemo* under github.com/rajeshsola/node-red-addons. I am planning to add a few more nodes for the historical data of *opensors.io*, which can be found in the above repository.

A few tips before you start to write custom nodes

1. Do a simple search in *flows.nodered.org* to avoid overlapping or duplication. You can merge your changes into existing nodes instead of adding new ones, unless your node has some distinguishing features or coding compared to the existing ones.
2. If not many configuration changes are required compared to available nodes, you can create example flows or sub-flows, e.g., most of the REST APIs and MQTT operations are possible with existing http-request and MQTT nodes (instead of adding new ones), or a simple function node may be good enough in between.
3. Once the new node is published to npm, new nodes will be pulled periodically and will be visible in the above library. Announce your contribution in the Node-RED mailing list for suggestions and feedback from the community. 

References

- [1] nodered.org/docs/, flows.nodered.org
- [2] nodered.org/docs/creating-nodes/
- [3] opensors.io

By: Rajesh Sola

The author is a faculty member of C-DAC's Advanced Computing Training School, and an evangelist in the embedded systems and IoT domains. You can reach him at rajeshsola@gmail.com.